

BİLİSİM ENSTİTUSU · YÜKSEK LİSANS

Nesneye Yonelik Yazilim Gelistirme

Ara Sinav · Cevap Belgesi

DONEM	2025-2026 Bahar
TESLİM TARİHİ	30 Nisan 2026
TOPLAM PUAN	100 (Soru 1: 50 + Soru 2: 50)
FORMAT	PDF, A4, tek dosya

Bu belge, sinavda istenen iki senaryo için nesneye yonelik tasarim cozumlerini sunar. Her soru için (i) senaryo cozumlemesi, (ii) tasarim oruntusu secimleri ile *red edilmiş alternatiflerin gerekceleri*, (iii) UML 2.x sinif diyagrami, (iv) bir veya iki sira (sequence) diyagrami, (v) onemli metotların sozde-kodları ve (vi) tipik çalışma akısları yer almaktadır. Diyagramlar, bağlilık (coupling) ve uyum (cohesion) olcutleri ile *okunaklılık* dengesi gozetilerek elle dizayn edilmiş; otomatik olarak yerlestirilmemistir.

İCİNDEKİLER

Soru 1 · Akilli Ev Yonetim Sistemi	State + Mediator + Observer
Soru 2 · SimpleCar E-Satis Sistemi	Strategy + CoR + Command

SORU 1 · 50 PUAN

Akilli Ev Yonetim Sistemi

State, Mediator ve Observer Oruntulerinin Birlikte Kullanimi

1.1 Senaryonun Cozumlenmesi

Senaryo, dort tip bilezen tanimlar: iki *aktif sistem* (aydinlatma, isitma), iki *pasif olcum kaynagi* (kapi sensoru, hareket sensoru) ve bir *orta kontrol noktasi* (akilli ev izleme motoru). Bilezenler arasindaki etkilesimleri dort temel olguya indirgeyebiliriz:

- Davranissal cogulluk.** Aydinlatma ve isitma sistemleri, ic durumlarına — yani aktif *moda* — gore *farkli davranir*. Tasarruflu modda isitma 18°C, aydinlatma asgari seviye hedefler; guvenli modda ise sirasiyla 40°C panel sicakligi ve maksimum aydinlatma uygulanir. Bu davranissal cogullugu **if/switch** ile cozmek, her yeni mod eklendiginde mevcut sinifi degistirmeyi gerektirir. Polimorfizm uzerinden tasinmasi *acik-kapali* prensibinin (OCP) dogal sonucudur.
- Kosullu senkronizasyon.** Iki sistemin modu *baglantilidir*: bir sistemde tasarruflu mod secilirse diger de zorunlu olarak gecer. Bu kural sistemler arasinda *dogrudan* uygulanirsa, her sistem digerini referans alir; $n \times (n-1)$ baglantisi ortaya cikar. Senaryoda iki sistem var, ancak ileride yeni bir sistem (orn. sulama) eklendiginde dogrudan referans yapisi kirilgan hale gelir.
- Olay yayini, alici farkindaligi olmadan.** Sensorler, olcumlerini iletmeli; ama *kime* ilettiklerini *bilmek zorunda olmamali*. Sensorun motora dogrudan referansi olmasi, sensoru izleme altyapisina baglar — sensoru baska bir baglamda (orn. test, ayri bir konsol) kullanmayi imkansizlastirir.
- Cogul bildirim hedefi.** Bir kullanicinin *birden fazla* istemcisi olabilir (mobil ve web). Motor, "kullaniciya bildirim gonder" eylemini somut bir istemci tipine baglarsa, yeni bir kanal (e-posta, SMS, sesli asistan) eklendiginde motor degisir. Burada da *yayinci-abone* ayrismasi kacinilmazdir.

1.2 Oruntu Secimleri ve Gerekceleri

Yukaridaki dort olgu, GoF kataloundaki uc *davranissal* oruntu ile noktasal olarak eslesir:

ORUNTU	UYGULANAN YER	COZDUGU OLGU
State <i>GoF, s.305</i>	<code>CalismaModu</code> arayuzu; <code>GuvenliMod</code> ve <code>TasarrufluMod</code> ; <code>EvSistemi</code> «baglam» rolunde, <code>aktifMod</code> alanini tutar.	(1) Davranissal cogulluk: <code>komutCalistir</code> cagirildiginda davranis aktif moda gore polimorfik secilir. Yeni bir mod eklemek <code>EvSistemi</code> 'ni degistirmez.

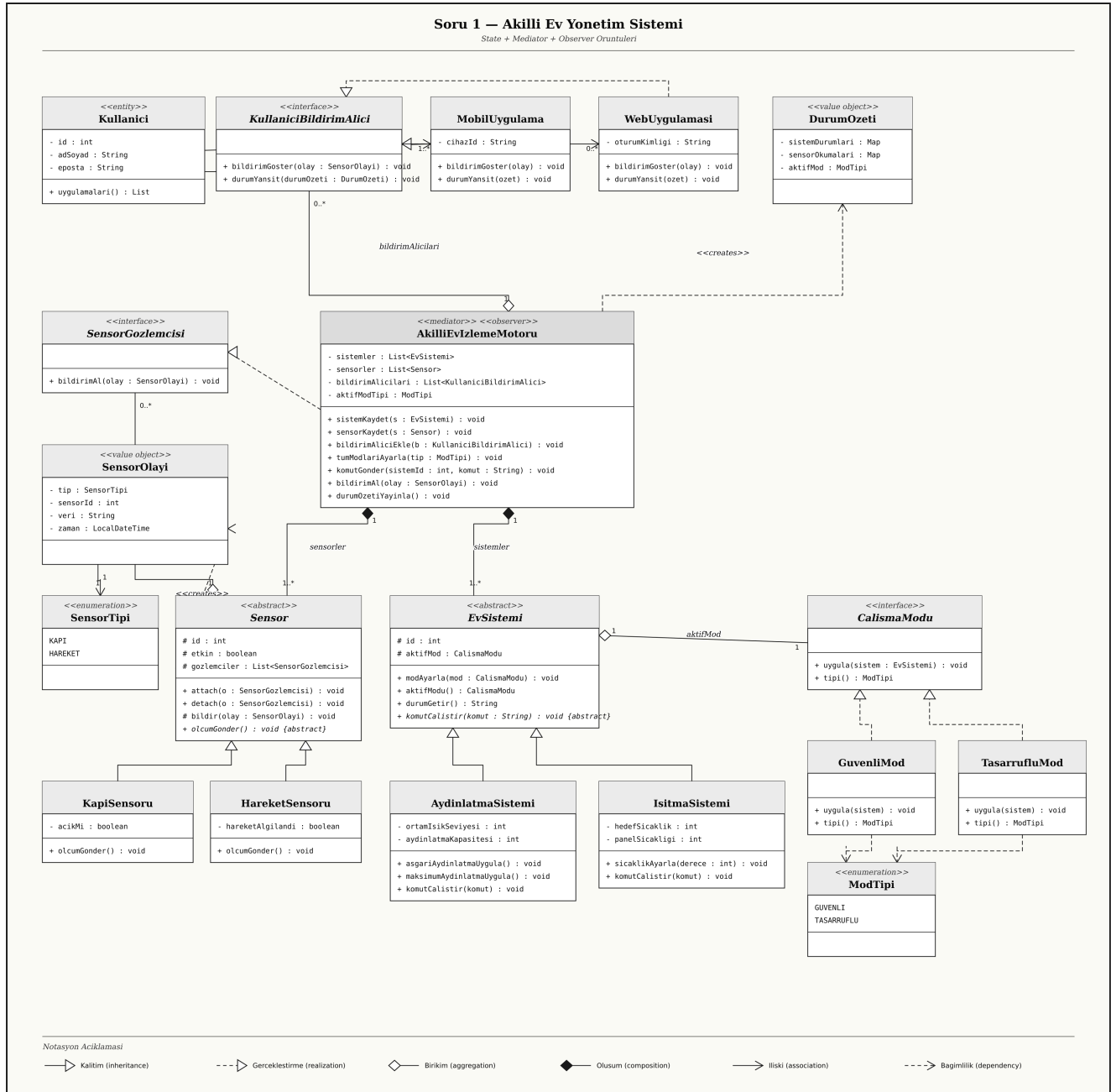
ORUNTU	UYGULANAN YER	COZDUGU OLGU
Mediator <i>GoF, s.273</i>	AkıllıEvIzlemeMotoru ; tum EvSistemi ve Sensor nesnelerini tanir, sistemleri birbirine baglamadan koordine eder.	(2) Kosullu senkronizasyon: "tum sistemler ayni moda gecsin" gibi cok-yonlu kurallar arabulucuda <i>tek bir noktada</i> uygulanir; sistemler arasi referans gerekmez.
Observer <i>GoF, s.293</i>	(a) Sensor ↔ SensorGozlemcisi arayuzu (motor abonedir). (b) AkıllıEvIzlemeMotoru ↔ KullaniciBildirimAlici arayuzu (mobil/web abonedir).	(3) Olay yayini ve (4) cogul bildirim hedefi: hem sensor hem motor, abone tipinden bagimsiz olarak yayin yapar. Yeni kanal/abone eklemek mevcut yayinciye degistirmez.

1.3 Reddedilen Alternatifler

ALTERNATİF	NEDEN TERCİH EDİLMEDİ?
Singleton — AkıllıEvIzlemeMotoru 'nu tekil yapmak.	Senaryo "bir akıllı ev" den bahsediyor; ancak <i>test edilebilirlik</i> ve <i>gelecek cok-evli</i> (apartman, site) senaryosu icin tekillik bagimlilikli maliyetli. Tekillik bir <i>uretim/yaratim</i> kararı; mevcut tasarim <i>davranissal</i> sorunlari cozer.
Strategy — modlari Calisma Stratejisi olarak tasimak.	Strategy, davranisi <i>disaridan</i> secilen bir algoritma olarak modeller; biz icine ait <i>durumu</i> modelliyoruz. Ozellikle "iki sistemin modu birlikte degisir" kurali, sistemden ayri bir strateji nesnesini gerektirmez. State daha dogru kavramsal eslesmedir.
Pub/Sub mesaj kuyruğu — sensor olaylari icin.	Senaryo bunu istemez; ek altyapi maliyeti getirir. Observer, <i>ayni surec icinde</i> ihtiyac duyulan ayrismayi sade bicimde saglar.
Visitor — mod degisikliginde sistemleri dolasmak icin.	Visitor, <i>sabit nesne hiyerarsisi uzerinde degisken islemler</i> icin uygundur. Burada islem (mod degistirme) sabit, hiyerarsi (sistemler) genisleyebilir; tam ters yon. Mediator + State daha uygun.

Mimari kanaat. *State davranisi, Mediator koordinasyonu, Observer ise bilgi yayinini bagimsiz eksenler olarak ayirir. Uc oruntu birbirinin kaldiraci: birini cikartmak diger ikisinin yukunu artirir ve kodun acilan/kapanan dengesini bozar.*

1.4 Sinif Diyagrami



Sekil 1.1 · Akilli Ev Yonetim Sistemi — UML 2.x sinif diyagrami. Sol katman: kullanıcı ve goruntuleme; orta katman: arabulucu motor ve gozlemci arayuzleri; alt katman: sistem ve sensor biyerarsileri.

1.5 Sinif Aciklamalari

SINIF / ARAYUZ	STEREOTİP	SORUMLULUK & ROLLER
Kullanici	<i><<entity>></i>	Ev sahibinin kimligi; bir veya birden fazla bildirim aliciya (mobil/web) sahip olabilir.
KullaniciBildirimAlici	<i><<interface>></i>	Observer arayuzu (kanal-bagimsiz). bildirimGoster(olay) ve durumYansit(ozet).

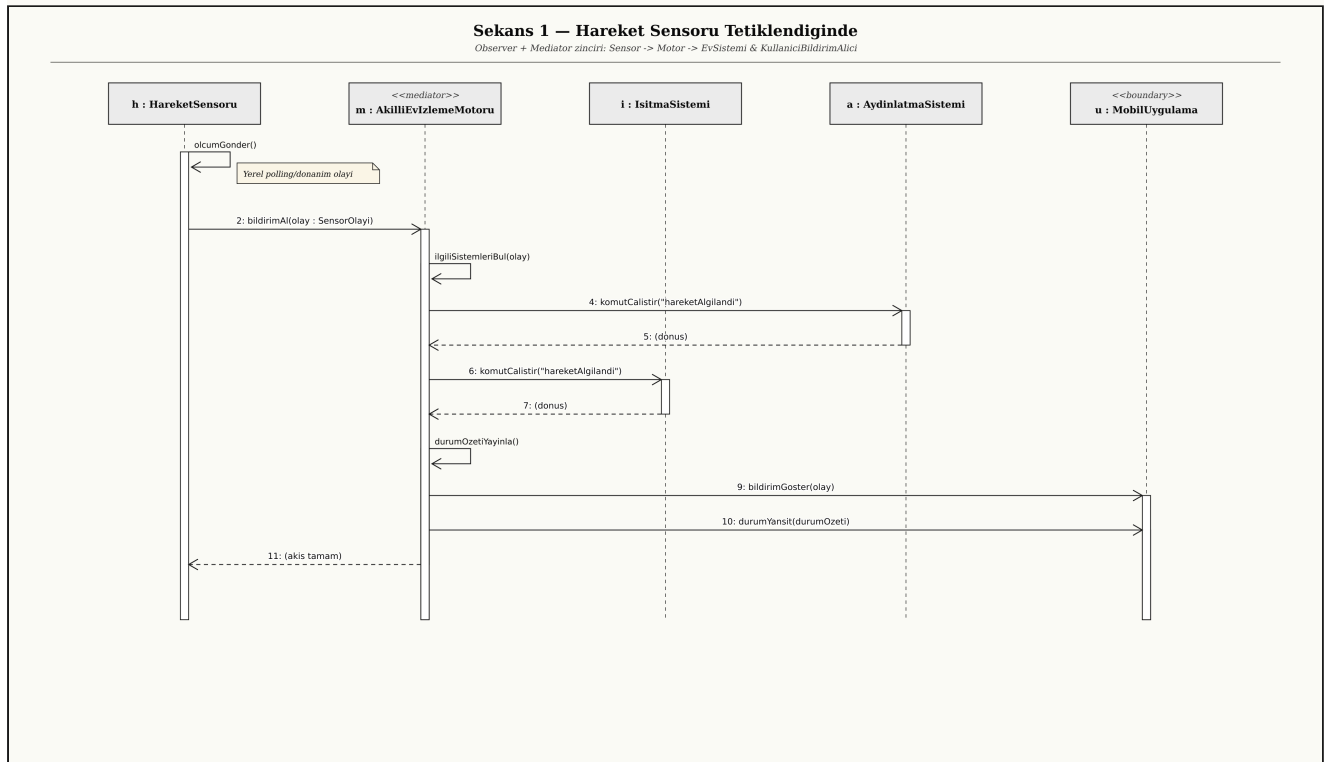
SINIF / ARAYUZ	STEREOTİP	SORUMLULUK & ROLLER
MobilUygulama . WebUygulamasi	«concrete»	Concrete observer; kanal-ozel goruntuleme yapar. Kullanici komutlarini motora iletir.
DurumOzeti	«value object»	Anlik durum fotografi; degismez (immutable). Motor uretir, alicilar tuketir.
AkilliEvIzlemeMotoru	«mediator» + «observer»	Tum sistem ve sensorleri tanir. Sensor olaylarinda gozlemci, kullanıcı için yayinci. Mod senkronizasyonu, komut yonlendirme ve durum yayini.
EvSistemi	«abstract»	State için <i>baglam</i> ; aktifMod alani üzerinden davranisini polimorfik secer. komutCalistir alt siniflara birakilir.
AydinlatmaSistemi . IsitmaSistemi	«concrete»	Cihaz-ozel davranisi (asgari/maksimum aydinlatma; hedef sicaklik) uygular.
CalismaModu	«interface»	State arayuzu; uygula(sistem) ile baglami yapilandirir.
GuvenliMod . TasarrufluMod	«concrete»	İki politik davranisi: maksimum kapasite ile asgari/hedef ayar.
ModTipi	«enumeration»	Kullanici arayuzunden gelen mod tercihi için tip-guvenli sabitler.
Sensor	«abstract»	Subject rolu: attach / detach ve bildir ortak metotlari.
KapiSensoru . HareketSensoru	«concrete»	Donanima ozgu olcum metotlari (olcumGonder); olay tetiklerler.
SensorGozlemcisi	«interface»	Sensor olaylarini alacak nesneler için Observer arayuzu (motor uygular).
SensorOlayi	«value object»	Tip, kaynak sensor, veri ve zaman tasiyan degismez olay nesnesi.
SensorTipi	«enumeration»	KAPI / HAREKET; mantik kontrolu ve kayit için.

1.6 İlişki Tipleri (UML Notasyonu)

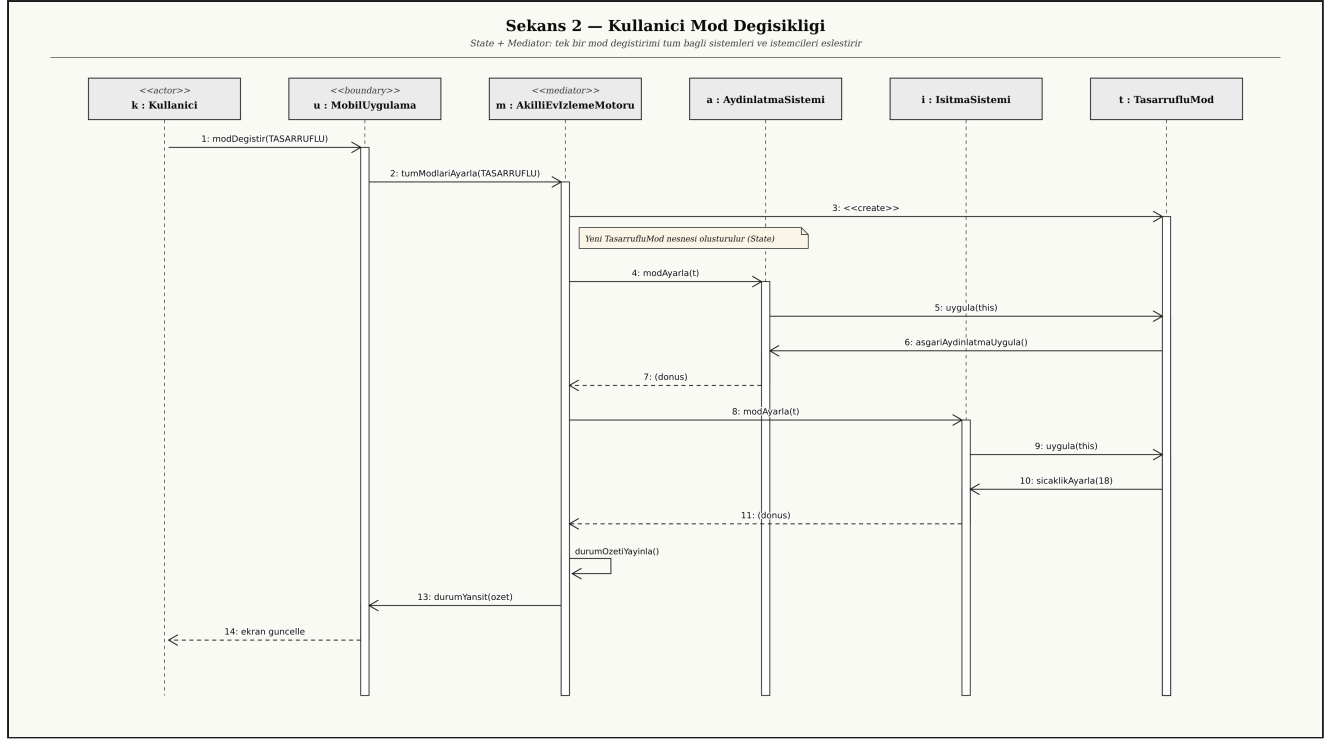
- Olusum (composition).* AkilliEvIzlemeMotoru ◆— EvSistemi (1..*) ve AkilliEvIzlemeMotoru ◆— Sensor (1..*) : motor olmadan sistem/sensor anlamlı değildir — yasam dongulerini motor sahiplenir.

- *Birikim (aggregation)*. AkilliEvIzlemeMotoru ◊— KullaniciBildirimAlici (0..*) : bildirim alicilari motorun parçasi degildir, dis dunyaya aittir; motor onlari yalnızca tutar.
- *Birikim (state)*. EvSistemi ◊— CalismaModu (aktif mod): mod nesnesi sistemin yasam dongusunden bagimsiz olusturulup atanir.
- *Gerçekleştirme (realization)*. MobilUygulama / WebUygulamasi —..▷ KullaniciBildirimAlici ; GuvenliMod / TasarrufluMod —..▷ CalismaModu ; AkilliEvIzlemeMotoru —..▷ SensorGozlemcisi .
- *Kalitim (inheritance)*. Sensor ve EvSistemi hiyerarsileri.
- *Bagimlilik (dependency)*. AkilliEvIzlemeMotoru —..> DurumOzeti («creates»): motor durum ozetini olusturup yayinlar; alicilar bunu yalnızca tuketir.

1.7 Sira Diyagramlari (Calisma Akislari)



Sekil 1.2 · Hareket sensoru tetiklendiginde olay akisi. Sensor olayi gozlemci uzerinden motora iletilir; motor ilgili sistemleri uyarir, sonra DurumOzeti ile tum bildirim alicilarini eslestirir.



Sekil 1.3 · Kullanicinin tasarruflu mod talebi. Tek bir **tumModlariAyarla** cagrisi, mediator araciligi ile tum sistemleri ve istemcileri tutarli bicimde gunceller; sistemler birbirini referans almaz.

1.8 Onemli Metotlarin Sozde-Kodu

```
// MEDIATOR – tum sistemleri tek noktadan moda gecir
class AkilliEvIzlemeMotoru implements SensorGozlemcisi {
    void tumModlariAyarla(ModTipi tip) {
        CalismaModu yeniMod = (tip == ModTipi.TASARRUFLU)
                               ? new TasarrufluMod()
                               : new GuvenliMod();
        for (EvSistemi s : sistemler) {           // baglantili sistemler
            s.modAyarla(yeniMod);                 // ayni nesne paylasilir
        }
        aktifModTipi = tip;
        durumOzetiYayinla();                     // observer'lara bildir
    }

    // OBSERVER – sensorden olay
    void bildirimAl(SensorOlayi olay) {
        for (EvSistemi s : ilgiliSistemleriBul(olay))
            s.komutCalistir(olay.tip == SensorTipi.HAREKET
                           ? "hareketAlgilandi" : "kapiAcildi");
        for (KullaniciBildirimAlici a : bildirimAlicilari)
            a.bildirimGoster(olay);              // kanal-bagimsiz yayin
        durumOzetiYayinla();
    }

    private void durumOzetiYayinla() {
        DurumOzeti ozet = new DurumOzeti(/* anlik durum */);
        for (KullaniciBildirimAlici a : bildirimAlicilari)
            a.durumYansit(ozet);
    }
}
```



```
// STATE – davranisi mod üzerinden polimorfik tasi
abstract class EvSistemi {
    protected CalismaModu aktifMod;
    void modAyarla(CalismaModu m) { this.aktifMod = m; m.uygula(this); }
    abstract void komutCalistir(String komut);
}

class IsitmaSistemi extends EvSistemi {
    void sicaklikAyarla(int derece) { /* donanim cagrisi */ }
    void komutCalistir(String komut) {
        // mod kararinin tum yuku CalismaModu.uygula icinde toplanmistir;
        // burada yalnızca komutu donanima yansitiriz
    }
}

class TasarrufluMod implements CalismaModu {
    void uygula(EvSistemi s) {
        if (s instanceof IsitmaSistemi) ((IsitmaSistemi)
s).sicaklikAyarla(18);
        if (s instanceof AydinlatmaSistemi) ((AydinlatmaSistemi)
s).asgariAydinlatmaUygula();
    }
    ModTipi tipi() { return ModTipi.TASARRUFLU; }
}
```

1.9 Tasarimin Niteliksel Avantajlari

- **Acik-Kapali (OCP).** Yeni mod (UykuModu), yeni sensor (Pencere), yeni bildirim kanali (E-posta); her durumda *yalnızca yeni bir sinif* eklenir, mevcut kod degismez.
- **Tek Sorumluluk (SRP).** Mod davranisi `CalismaModu` hiyerarsisinde; senkronizasyon politikasi `AkilliEvIzlemeMotoru` 'nda; olay tasimasi `Sensor` + `SensorGozlemcisi` ikilisinde tutulmustur.
- **Dusuk Baglilik.** Sistemler birbirini, sensorler de motoru somut bir tip olarak bilmez.
- **Test Edilebilirlik.** Stub gozlemci ve sahte modlar ile motor birim test seviyesinde dogrulanabilir.
- **Liskov Uyumu.** Yeni mod, yeni sistem ve yeni sensor turleri ust sozlesmenin (*contract*) ihlali olmadan eklenir.

SORU 2 · 50 PUAN

SimpleCar E-Satis Sistemi

Strategy, Chain of Responsibility ve Command Oruntulerinin Birlikte Kullanimi

2.1 Senaryonun Cozumlenmesi

SimpleCar firmasi yalnızca e-satis yapan bir araba ureticisidir. Domeni dort bilesene ayrilabilir:

- 1. Algoritma ailesi (odeme).** Odeme tek bir *davranistir*, ancak *banka havalesi*, *paypal*, *soguk cuzdan* gibi farkli yollarla yapilir; senaryo ayrica bu listenin *siklikla genisleyip daralacagini* ozellikle vurgular. Calisma zamaninda secilebilen, bagimsiz olarak degistirilebilen bir *algoritma ailesi* gerekir.
- 2. Asamali ve genisleyebilir kontrol.** Her siparis *sahtecilik*, *limit* ve *bakiye* kontrollerine tabidir. Birinin olumsuzlugu siparisi iptal eder. Senaryo bu kontrollerin de zamanla *artirilabilecegini* belirtir. Bu, bir *filtre zinciri*: her halka kararini verir, basariliysa siradakine devreder; basarisizsa zinciri keser.
- 3. Sirali isleme adimlari.** Tum kontroller basariliysa siparis sirasiyla *fatura duzenleme*, *fatura gonderme* ve *alacaklara kaydetme* adimlariyla islenir. Adimlar ileride degisebilir; her biri test edilebilir bagimsiz bir is birimini temsil eder.
- 4. Yonetim ve yasam dongusu.** Bir orchestrator (use-case) bilesen, siparisi *alir*, *dogrular* ve *isler*. Domain (Siparis) ile servis sorumluluklari ayrik tutulmalidir.

2.2 Oruntu Secimleri ve Gerekceleri

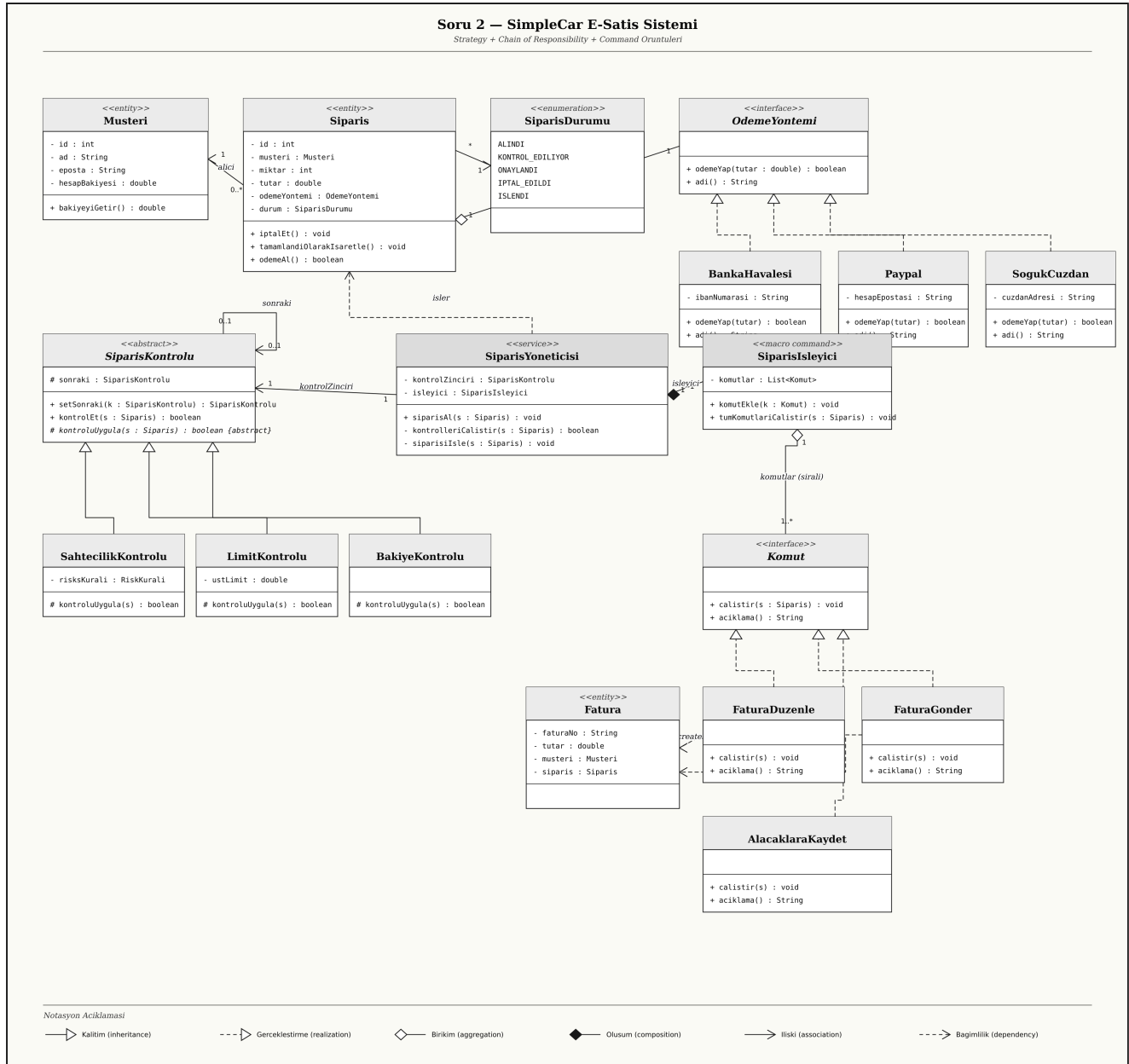
ORUNTU	UYGULANAN YER	COZDUGU OLGU
Strategy <i>GoF, s.315</i>	<code>OdemeYontemi</code> arayuzu; <code>BankaHavalesi</code> , <code>Paypal</code> , <code>SogukCuzdan</code> ; siparisle 1-1 birikim.	(1) Algoritma ailesi: odeme ailesi siparisten bagimsiz olarak yer degistirebilir; yeni yontem eklemek mevcut kodu degistirmez.
Chain of Responsibility <i>GoF, s.223</i>	<code>SiparisKontrolu</code> soyut sinifi; <code>Sahtecilik</code> , <code>Limit</code> , <code>Bakiye</code> halkalari; <code>setSonraki</code> ile kuruluyor.	(2) Asamali kontrol: halka basariliysa siradakine devreder, basarisizsa zincir kesilir; yeni kontrol eklemek tek satir konfigurasyondur.
Command + Macro Command <i>GoF, s.233</i>	<code>Komut</code> arayuzu; <code>FaturaDuzenle</code> , <code>FaturaGonder</code> , <code>AlacaklaraKaydet</code> ; <code>SiparisIsleyici</code> sirayla calistirir.	(3) Sirali isleme adimlari: her adim kapsullenmis; sira degistirme, ek adim, geri-alma (undo) gibi ihtiyaclar icin esnek altyapi.

2.3 Reddedilen Alternatifler

ALTERNATİF	NEDEN TERCİH EDİLMEDİ?
Template Method — siparis isleme akisini soyut sinifta sablon olarak kurmak.	Sablon, <i>sıra sabit</i> ise iyidir; ancak senaryo "adim sayisi degisebilir" implikasyonunu tasir (yeni adimlar). Komut listesi, calisma zamaninda yeniden duzenlenebilir; sablon ise siniflarin yeniden yazilmasini gerektirir.
Decorator — kontrolleri sarmalayici olarak eklemek.	Decorator, ana arayuze yeni davranis ekler; ancak burada kontroller <i>kararsal</i> : false donerse akisi durdurur. Bu, "ekleme" degil "eleme" semantigi; CoR daha dogru.
Observer — siparis durumu degistikce abonelere haber vermek.	Faydali olabilirdi; ancak senaryo "bilgilendirme" istemiyor, <i>islem akisini</i> tarif ediyor. Cozumumuz islem-yonludur; Observer ihtiyaca gore daha sonra eklenebilir, mimariyi engellemez.
State — siparis durumu icin.	<code>SiparisDurumu</code> bir enum yeterli; siparisin davranisi durumla degismiyor (yalnizca basit gozlemlenebilir alanlar). State asiri muhendislik olur.

Mimari kanaat. *Strategy "ne ile odensin", CoR "kabul edilebilir mi", Command "kabul sonrasi ne yapilsin" sorularini birbirinden bagimsiz cevaplar. Senaryonun her uc degisme ekseni — odeme yontemleri, kontrol kurallari, isleme adimlari — ayri bir oruntu ile soyutlanmistir; biri buyurken digerleri sabit kalir.*

2.4 Sinif Diyagrami



Sekil 2.1 · SimpleCar E-Satis — UML 2.x sinif diyagrami. Sol kanat: CoR (kontroller); orta: orchestrator; sag ust: Strategy (odeme yontemleri); sag alt: Command (sirali isleme adimlari).

2.5 Sinif Aciklamalari

SINIF / ARAYUZ	STEREOTİP	SORUMLULUK & RÖLER
Musteri	«entity»	Siparisi veren kisi; hesapBakiyesi bakiye kontrolunde kullanilir.
Siparis	«entity»	Aggregate root; tutar, miktar, secilen OdemeYontemi, durum bilgilerini tasir.
SiparisDurumu	«enumeration»	Yasam dongusu sabitleri: ALINDI / KONTROL_EDILIYOR / ONAYLANDI /

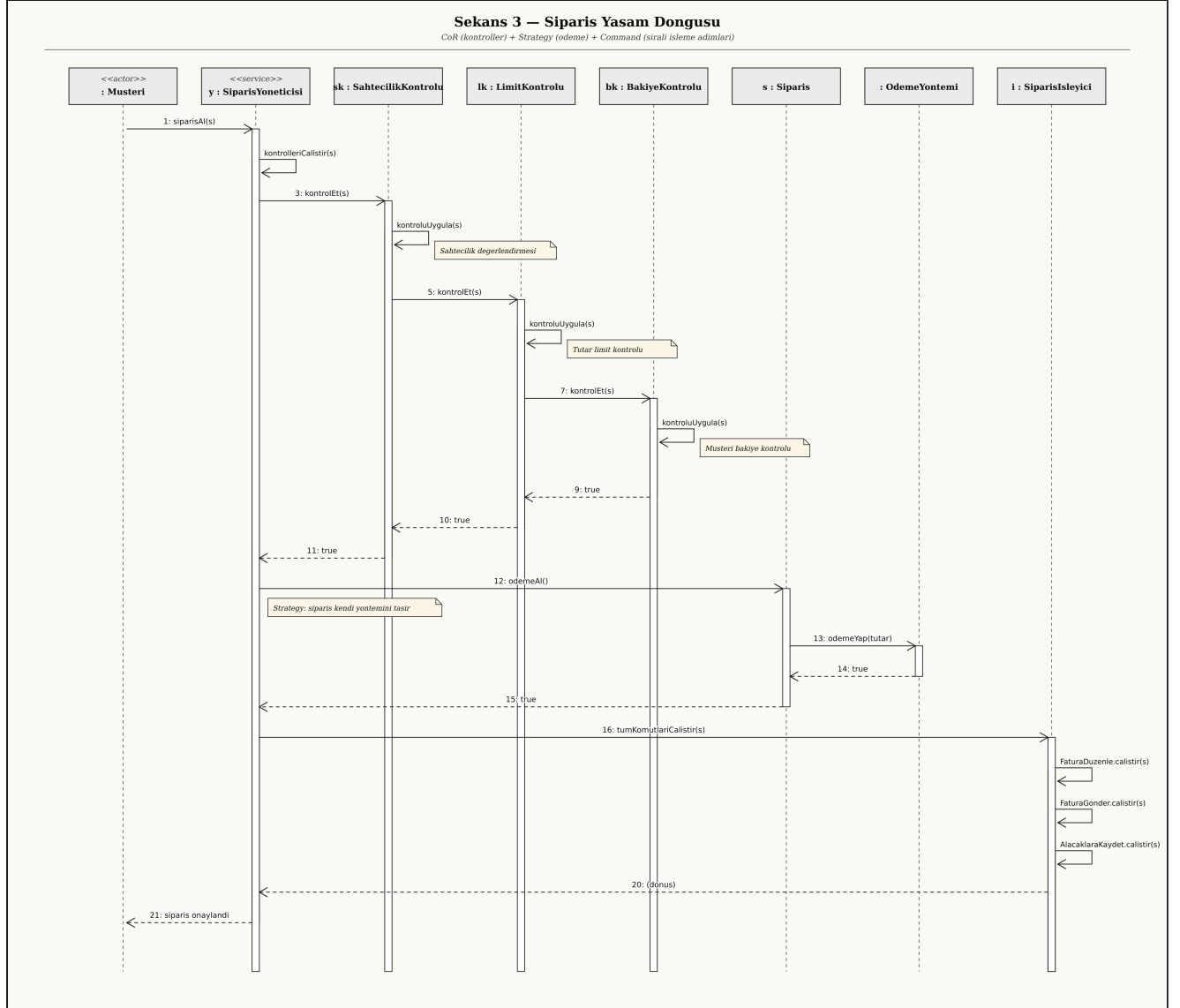
SINIF / ARAYUZ	STEREOTİP	SORUMLULUK & ROLLER
		IPTAL_EDILDI / ISLENDI.
OdemeYontemi	«interface»	Strategy sozlesmesi; <code>odemeYap(tutar)</code> imzasi.
BankaHavalesi . Paypal . SogukCuzdan	«concrete»	Mevcut odeme stratejileri; her birinin kanal-ozel alanlari (IBAN, e-posta, cuzdan adresi) vardir.
SiparisKontrolu	«abstract»	CoR taban sinifi; <code>kontrolEt</code> sablon metodu zincirleme isini tasir, <code>kontrolUygula</code> alt siniflara birakilir.
SahtecilikKontrolu . LimitKontrolu . BakiyeKontrolu	«concrete»	Tek bir kurali uygulayan halkalar; bagimsiz birim test yapılabilir.
Komut	«interface»	Command sozlesmesi; <code>calistir(siparis)</code> .
FaturaDuzenle . FaturaGonder . AlacaklaraKaydet	«concrete»	Senaryoda istenen sirali uc isleme adimi; her biri kendi yan etkisinden sorumlu.
SiparisIsleyici	«macro command»	Sirali komut listesi; <code>komutEkle</code> ile genisletilebilir.
SiparisYoneticisi	«service»	Use-case orchestratoru; kontrol zincirini ve isleyiciyi koordine eder. <i>Acik:</i> burada Facade kalitsal bir izi vardir — cagirana tek bir <code>siparisAl(s)</code> arayuzu sunulur.
Fatura	«entity»	Fatura komutlari arasinda paylasilan veri; siparis ile 1-1 iliskili.

2.6 Iliski Tipleri (UML Notasyonu)

- Birikim (aggregation).* `Siparis` ◊- `OdemeYontemi` (1..1) : odeme yontemi siparise atanir, fakat onun parçasi degildir.
- Birikim.* `SiparisIsleyici` ◊- `Komut` (1..*, sirali) : komut listesi isleyiciden bagimsiz olarak da var olabilir.
- Olusum (composition).* `SiparisYoneticisi` ◆- `SiparisIsleyici` : yoneticisi, isleyiciyi kendisi olusturur ve sahiplenir.
- Refleksif iliski.* `SiparisKontrolu` — sonraki: 0..1 : zincirleme bagintisi, halkanın kendisine.
- Gerçekleştirme (realization).* `BankaHavalesi/Paypal/SogukCuzdan` —..▷ `OdemeYontemi` ;
`FaturaDuzenle/FaturaGonder/AlacaklaraKaydet` —..▷ `Komut` .
- Kalitim.* Uc kontrol halkasi `SiparisKontrolu` 'ndan turer.

- *Bagimlilik.* SiparisYoneticisi –.> Siparis («invokes»); FaturaDuzenle –.> Fatura («creates»).

2.7 Sira Diyagrami (Siparis Yasam Dongusu)



Sekil 2.2 · Bir siparin alinmasindan ISLENDI durumuna gecisine kadar tum etkilesim akisi. Kontroller CoR boyunca devredilir; isleme adimlari Macro Command tarafindan sirayla calistirilir.

2.8 Onemli Metotlarin Sozde-Kodu

```
// CHAIN OF RESPONSIBILITY – taban sinif
abstract class SiparisKontrolu {
    protected SiparisKontrolu sonraki;
    SiparisKontrolu setSonraki(SiparisKontrolu k) { this.sonraki = k; return k; }

    final boolean kontrolEt(Siparis s) {          // template method
        if (!kontrolUygula(s)) return false;    // basarisizsa zincir kesilir
        return sonraki == null ? true
            : sonraki.kontrolEt(s);
    }
    protected abstract boolean kontrolUygula(Siparis s);
}

class BakiyeKontrolu extends SiparisKontrolu {
    protected boolean kontrolUygula(Siparis s) {
        return s.musteri.bakiyeyiGetir() ≥ s.tutar;
    }
}
```

```
// MACRO COMMAND – sirali komut listesi
class SiparisIsleyici {
    private final List<Komut> komutlar = new ArrayList<>();
    void komutEkle(Komut k) { komutlar.add(k); }
    void tumKomutlariCalistir(Siparis s) {
        for (Komut k : komutlar) k.calistir(s);
    }
}

// USE-CASE ORCHESTRATOR
class SiparisYoneticisi {
    private final SiparisKontrolu kontrolZinciri;
    private final SiparisIsleyici isleyici;

    void siparisAl(Siparis s) {
        s.durum = SiparisDurumu.KONTROL_EDILIYOR;
        if (!kontrolZinciri.kontrolEt(s)) {
            s iptalEt();                      // durum = IPTAL_EDILDI
            return;
        }
        if (!s.odemeAl()) { s iptalEt(); return; } // strategy uzerinden
        s.durum = SiparisDurumu.ONAYLANDI;
        isleyici.tumKomutlariCalistir(s);      // fatura duzenle/gonder/alacak
        s.tamamlandiOlarakIsaretle();         // durum = ISLENDI
    }
}
```

```
// STRATEGY – siparisten secilen yontem cagirilir
class Siparis {
    private OdemeYontemi odemeYontemi;
    boolean odemeAl() { return odemeYontemi.odemeYap(this.tutar); }
}

class Paypal implements OdemeYontemi {
    private final String hesapEpostasi;
    public boolean odemeYap(double tutar) { /* ag cagrisi */ return true; }
    public String adi() { return "Paypal"; }
}
```

2.9 Genisletilebilirlik Senaryolari

YENİ GEREKSİNİM	MEVCUT TASARIMDA YAPILACAK TEK ŞEY
Kripto kart yontemi eklemek	<code>OdemeYontemi</code> 'ni uygulayan yeni bir sinif yazmak. Mevcut hicbir kod degismez.
KYC kontrolu eklemek	<code>SiparisKontrolu</code> 'ndan turetilen yeni bir halka yazip zincire <code>setSonraki</code> ile takmak.
Kargo etiketi olusturma adimi eklemek	<code>Komut</code> 'u uygulayan yeni bir sinifi <code>SiparisIsleyici.komutEkle</code> ile dahil etmek.
Bazi adimlari paralel calistirma	<code>SiparisIsleyici</code> 'yi <code>ParalelSiparisIsleyici</code> olarak alternatiflestirmek (Liskov uyumlu).
Tum kontrolleri tek seferde calistirip <i>tum</i> hatalari toplama	Yeni bir <code>HepsiniDogrulayanSiparisKontrolu</code> kompozit halkasi yazmak; mevcut halkalar degismeden kullanilir.

2.10 Sonuc

Onerilen tasarim; siparisin *alinmasi*, *dogrulanmasi* ve *islenmesi* ic akislarini birbirinden ayri, oruntu-tabanli kapsullere yerlestirir. Senaryonun en sik degisecegi belirtilen iki eksen — *odeme yontemleri* ve *kontrol kurallari* — mevcut kodu degistirmeden buyur. Sirali isleme, gozlemlenebilir, sıra/komut listesi degistirebilir ve test edilebilir bir komut akisi ile gerceklestirilir. Boylece, hem sinav kurallarinda istenen *oruntu adi acikca belirtilmis* ve *iliski tip ile isaretlemeleri dogru kullanilmis*; hem de senaryodaki nitelikler ve metotlar siniflar icinde konumlandirilmistir.